

INFORME TÉCNICO

Sistema de Facturación Electrónica Médica

Fecha de generación: 23/7/2025

Estado general del proyecto: 50% completado

RESUMEN EJECUTIVO

El proyecto de Sistema de Facturación Electrónica Médica se encuentra en un estado de desarrollo del 50%. La interfaz de usuario está prácticamente completa (90%), pero requiere implementación completa del backend, base de datos real, y integraciones de producción para ser funcional en un entorno real.

COMPONENTES CRÍTICOS PENDIENTES:

- Base de datos real (PostgreSQL/MySQL)
- Autenticación y autorización robusta
- Integración SOAP real con SRI
- Sistema de seguridad completo
- Tests automatizados

ANÁLISIS DETALLADO POR COMPONENTE

1. Interfaz de Usuario (Frontend)

Estado: 90% - COMPLETO

Descripción: Sistema completo de interfaces con Next.js 14, React 18, TypeScript y Tailwind CSS

Archivos principales:

- app/medico/dashboard/page.tsx
- app/paciente/formulario/page.tsx
- app/facturador/dashboard/page.tsx

Estado actual:

- ' Diseño responsive completamente implementado
- ' Componentes reutilizables con shadcn/ui
- ' Navegación fluida entre módulos
- ' Formularios con validación en tiempo real

2. Gestión de Estado y Cache

Estado: 75% - PARCIAL

Descripción: Sistema de cache en memoria con WebSocket simulado para tiempo real

Archivos principales:

- lib/cache/cache-manager.ts
- lib/cache/memory-cache.ts
- lib/websocket/websocket-client.ts

Estado actual:

```
' Cache L1 en memoria implementado
' WebSocket client simulado funcional
' Hooks personalizados para estado
'L Falta cache L2 (Redis/Database)
```

3. Autenticación y Autorización

Estado: 30% - CRITICO

Descripción: Sistema básico con localStorage, requiere implementación completa

Archivos principales:

- app/medico/login/page.tsx
- app/facturador/login/page.tsx
- middleware.ts (no existe)

Estado actual:

```
&p Solo validación frontend básica
'L Sin JWT o tokens seguros
'L Sin middleware de protección
'L Sin roles y permisos granulares
```

4. Base de Datos

Estado: 0% - CRITICO

Descripción: Actualmente usa localStorage, requiere base de datos real

Archivos principales:

- lib/database/* (no existe)
- prisma/* (no existe)

Estado actual:

```
'L Sin base de datos real
'L Sin ORM (Prisma/Drizzle)
'L Sin migraciones
'L Sin backup automático
```

5. Integración SRI (Facturación Electrónica)

Estado: 60% - PARCIAL

Descripción: Estructura completa pero con simulación, falta integración real

Archivos principales:

- lib/sri/sri-client.ts
- lib/sri/xml-generator.ts
- lib/sri/certificate-manager.ts

Estado actual:

- ' Generación de XML de facturas
- ' Gestión de certificados digitales
- ' Configuración de ambientes SRI
- ' Cálculo de clave de acceso

6. Sistema de Notificaciones

Estado: 70% - PARCIAL

Descripción: Notificaciones en tiempo real simuladas, falta backend real

Archivos principales:

- components/notifications/notification-center.tsx
- components/websocket/connection-status.tsx
- lib/websocket/types.ts

Estado actual:

- ' Centro de notificaciones UI
- ' Estados de conexión
- ' Notificaciones en tiempo real (simuladas)
- 'L Push notifications

7. Generación de Reportes

Estado: 40% - PARCIAL

Descripción: Reportes básicos implementados, falta generación de PDFs

Archivos principales:

- app/medico/reportes/page.tsx
- lib/reports/* (parcial)

Estado actual:

- ' Interfaz de reportes básica
- ' Datos estadísticos simulados
- 'L Generación de PDFs
- 'L Gráficos interactivos

8. Seguridad

Estado: 25% - CRITICO

Descripción: Seguridad básica implementada, requiere refuerzo completo

Archivos principales:

- lib/sri/certificate-manager.ts
- middleware.ts (falta)

Estado actual:

```
' Encriptación de contraseñas de certificados  
' Validación básica de formularios  
'L Rate limiting  
'L CSRF protection
```

RECOMENDACIONES PARA PRODUCCIÓN

PRIORIDAD ALTA (Crítico para funcionamiento):

1. Implementar base de datos PostgreSQL con Prisma ORM
2. Sistema de autenticación con NextAuth.js o similar
3. Integración SOAP real con servicios SRI de Ecuador
4. Implementar firma electrónica real con certificados
5. Sistema de logs y monitoreo (Winston + ELK Stack)

PRIORIDAD MEDIA (Calidad y seguridad):

6. Tests automatizados (Jest + Testing Library)
7. Rate limiting y protección CSRF/XSS
8. Generación de PDFs para facturas
9. Sistema de backup automático
10. Optimización de performance

PRIORIDAD BAJA (Mejoras futuras):

11. App móvil React Native
12. Dashboard de analytics avanzado
13. Sistema de pagos integrado
14. Notificaciones push
15. Multi-tenancy para múltiples consultorios

ESTIMACIONES DE DESARROLLO

Tiempo estimado para MVP en producción: 3-4 meses

Equipo recomendado: 1 Full-stack + 1 DevOps + 1 QA

Costo estimado de desarrollo: \$15,000 - \$25,000 USD

Costo mensual de infraestructura: \$200 - \$500 USD

TECNOLOGÍAS RECOMENDADAS:

- Base de datos: PostgreSQL + Prisma
- Autenticación: NextAuth.js + JWT
- Cache: Redis

- Monitoreo: Sentry + LogRocket
- Deploy: Vercel + Railway/PlanetScale
- CI/CD: GitHub Actions